

Statistical and Machine Learning SS26

Exercise Sheet 9

Hierarchical clustering

The `hclust()` function implements hierarchical clustering in R.

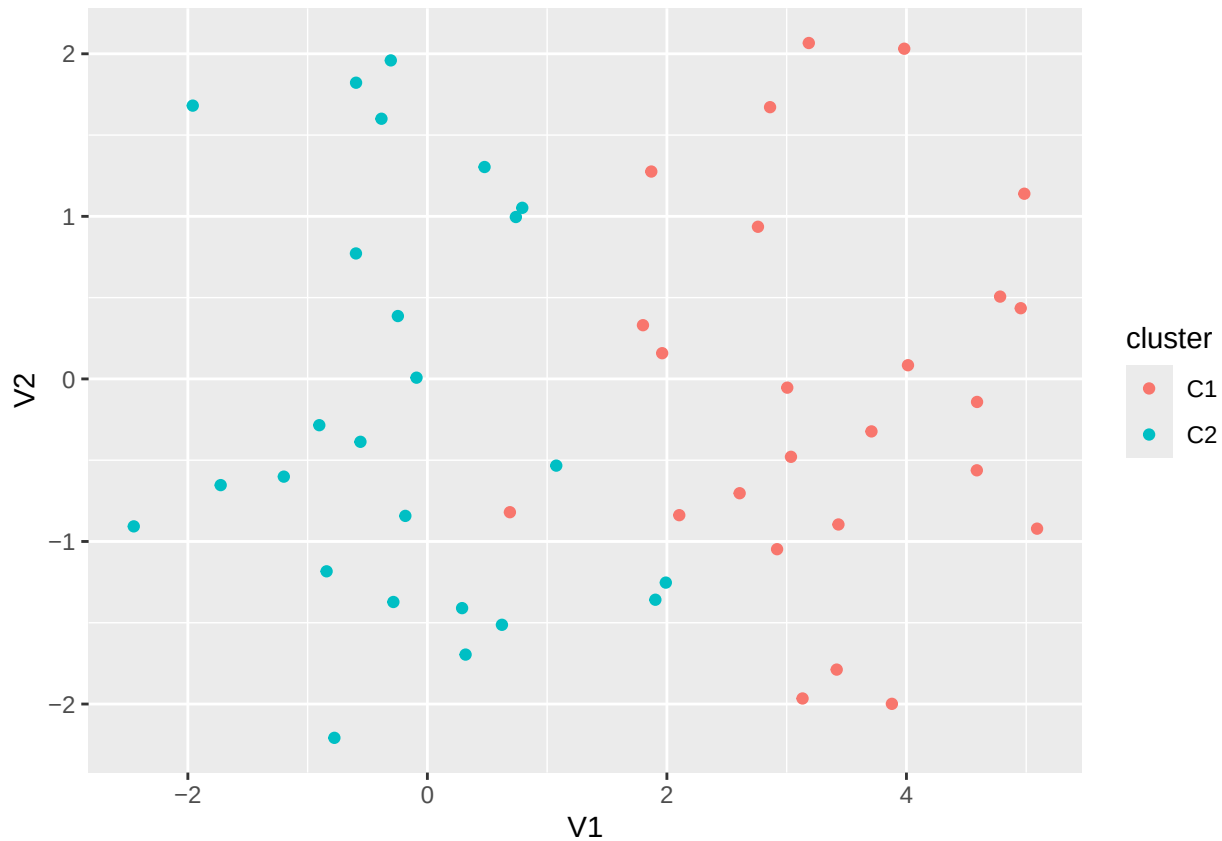
In the following example we plot the hierarchical clustering dendrogram using three different rules for measuring the distance between *clusters* (not individual points)

- a) complete linkage clustering: merges groups only if all members are close
- b) single linkage clustering: merges groups if any pair of members is close
- c) average linkage clustering: averages over all cross-group pairs

All of them are measured with Euclidean distance (most common choice - term for how we define 'distance' between points) as the dissimilarity measure.

```
suppressWarnings(suppressMessages(library(tidyverse))) #load package, hide info messages
set.seed(2)
x <- matrix(rnorm(50 * 2), ncol = 2)
# generate 100 random numbers from a normal distribution, arrange them into a 50x2 matrix
# 50 rows/observations, 2 columns (coordinates)
# like 50 points of a 2 dimensional plane, all clustered around (0,0)
x[1:25, 1] <- x[1:25, 1] + 3 #select column 1 of rows 1-25 and add 3
# -> shifts first 25 points to the right by 3 units
# by doing that we create two separate groups:
# 25 points around x ~ 3, 25 points around x ~ 0
x[1:25, 2] <- x[1:25, 2] #y coordinates are left as they are

plt_x <- as.data.frame(x) #converts matrix x into dataframe (ggplot2 requires df)
colnames(plt_x) <- c("V1", "V2") #name columns
plt_x$cluster = c(rep("C1", 25), rep("C2", 25)) # adds new column `cluster` that
# labels each point C1 or C2, depending on which group they belong to
ggplot(plt_x) + geom_point(aes(x = V1, y = V2, color = cluster))
```



*# plots 50 points as scatter plot, we can clearly see that cluster C1 is scattered around
 # V1 ~ 3, while C2 is scattered around V1 ~ 0. Both have a V2 mean of 0
 # (we didn't shift any cluster in y direction)*

Now everything we need has been set up and we can use the three different rules to measure the distance between clusters

a) Complete linkage clustering

Complete linkage (farthest neighbor linkage) determines the distance between two clusters as the largest distance between any two points in the two clusters

1. Build 50×50 distance matrix (Euclidean distances) → 50 clusters, each 1 point
2. Find smallest value in matrix → merge those two points (e.g. A, B) → update matrix: replace A and B with cluster (A,B) → $\text{dist}((A,B), C) = \max(\text{dist}(A,C), \text{dist}(B,C))$ (complete linkage rule)
3. Find smallest value in updated matrix → merge → update → $\text{dist}((A,B), C) = \max$ of all pairwise distances between cluster members
4. Repeat until 1 cluster remains
5. The full merge history = the dendrogram

<https://www.statisticshowto.com/complete-linkage-clustering/>

```
hc.complete <- hclust(dist(x), method = "complete")
# dist() function is used to compute the inter-observation Euclidean distance matrix
```

b) Single linkage clustering

Single linkage (nearest neighbor linkage) determines the distance between two clusters as the shortest distance between any two points in the two clusters

```
hc.single <- hclust(dist(x), method = "single")
```

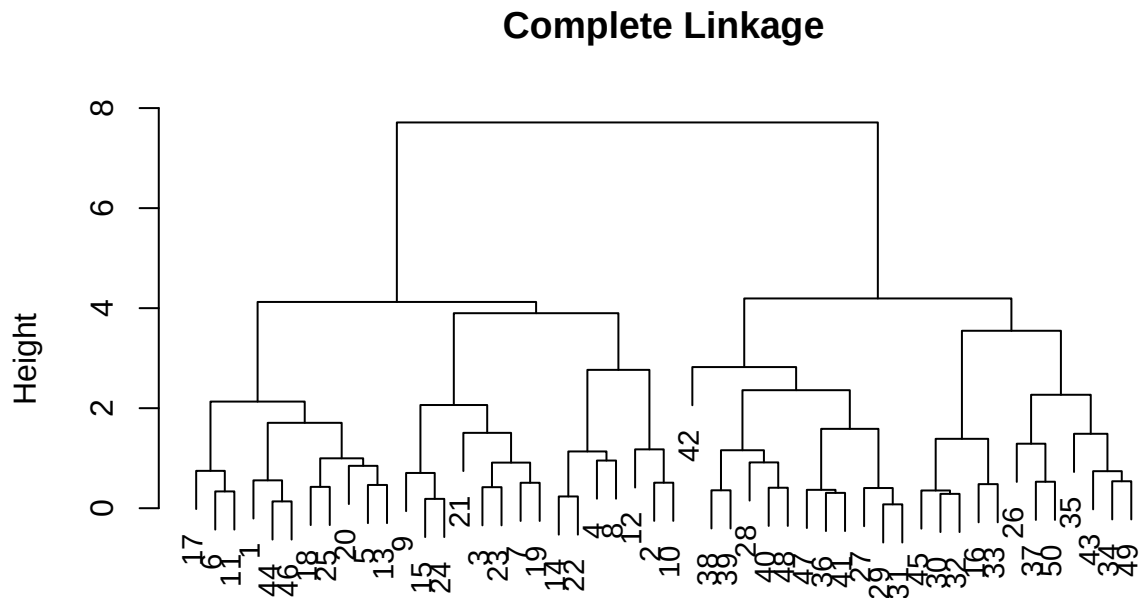
c) Average linkage clustering

Average linkage determines the distance between two clusters as the average distance between all pairs of points in the two clusters

```
hc.average <- hclust(dist(x), method = "average")
```

Plot the dendrograms. The numbers at the bottom of the plot identify each observation.

```
plot(hc.complete, main = "Complete Linkage", xlab = "", sub = "", cex = .9)
```



Complete Linkage:

Two clearly separated, compact clusters

The large gap between height 4 and 7.8 with no merges in between is exactly what we want to see:

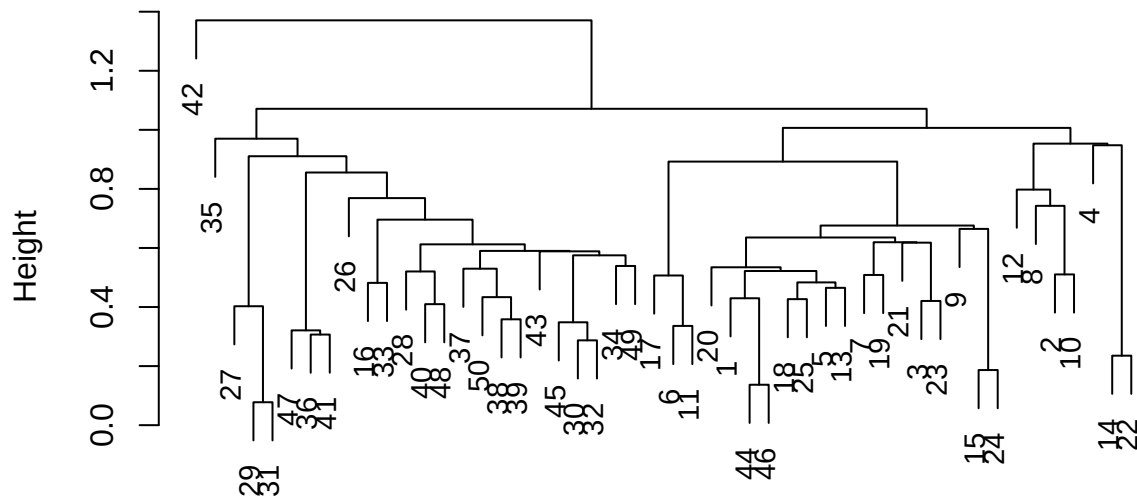
It confirms that cutting at 2 clusters is the right choice for this data

Left cluster: points 1-25, right cluster: points 26-50

BUT It got 3 wrong! 44, 46, 16

```
plot(hc.single, main = "Single Linkage", xlab = "", sub = "", cex = .9)
```

Single Linkage



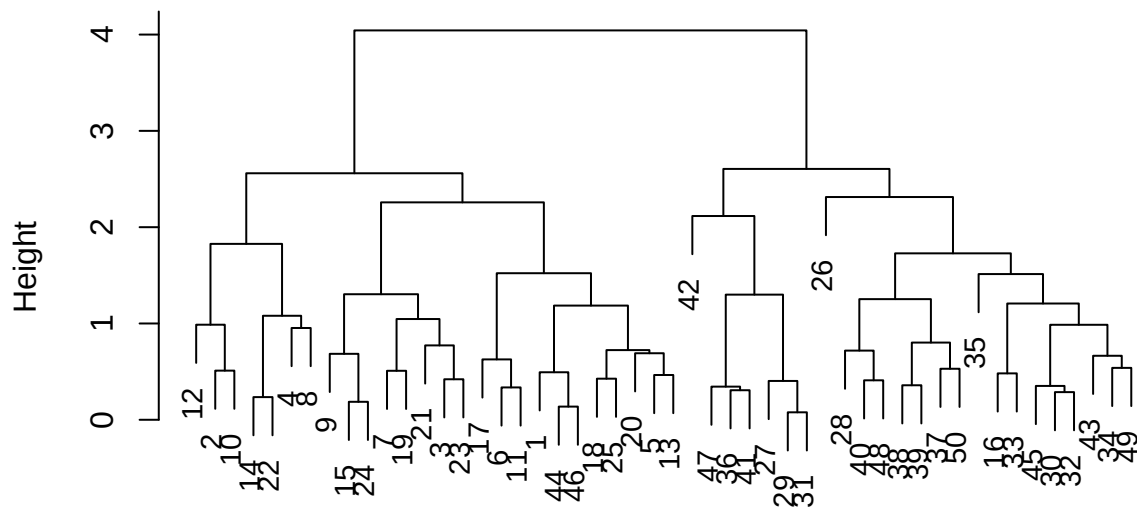
Single Linkage:

Much smaller overall heights because it always uses the minimum distance

Instead of two balanced groups forming independently, we see points being added one by one to a growing chain

```
plot(hc.average , main = "Average Linkage", xlab = "", sub = "",cex = .9)
```

Average Linkage



Average linkage sits right between complete and single linkage

It produces intermediate heights (makes sense since it uses the mean distance rather than max or min)

Final merge happens later than with complete linkage (4 instead of 7.8), but it still produces two clear clusters like complete linkage

Missclassifies 16, 44, 46 as well For this dataset, complete linkage gives the clearest and most interpretable result

To determine the cluster labels for each observation associated with a given cut of the dendrogram, we can use the ‘cutree()’ function: The second argument to ‘cutree()’ is the number of clusters we wish to obtain.

```
h_complete <- cutree(hc.complete,2)
h_complete
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 1 2 1 2 2 2 2
```

```
# complete linkage cluster labels: generally separates the observations into their correct groups.
# algorithm misclassified only 3 out of 50 points (16, 44, 46)
# These three points are likely sitting close to the boundary between the two groups
```

```
h_average <- cutree(hc.average , 2)
h_average
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 1 2 1 2 2 2 2
```

```
# average linkage misclassifies 16, 44, 46 as well - but correctly classifies the other 47 points!
```

```
h_single <- cutree(hc.single,2)
h_single
```

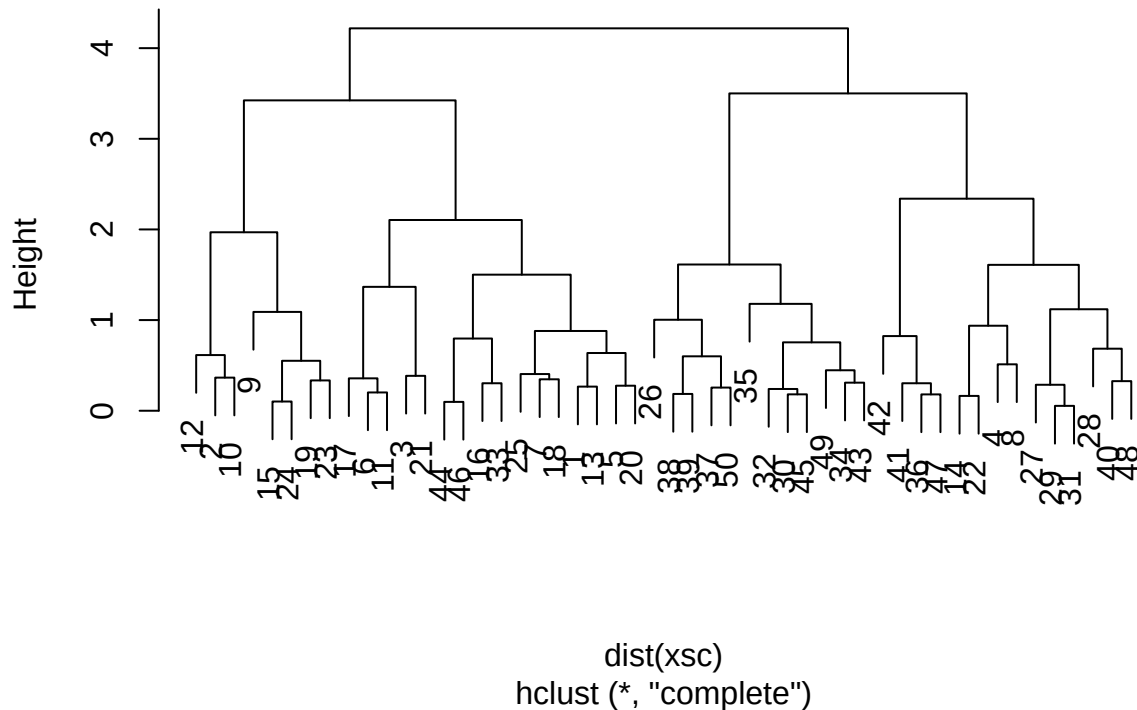
```
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [39] 1 1 1 2 1 1 1 1 1 1 1 1
```

```
# Single linkage identifies only one point as belonging to second cluster.
```

To scale the variables before performing hierarchical clustering of the observations, we use the ‘scale()’ function:

```
xsc <- scale(x) #standardizes every column to mean = 0 and sd = 1
# Standardizing is crucial in real world data because variables with larger values or
# larger variance would otherwise dominate the distance calculation and thus the clustering
hc.xsc <- hclust(dist(xsc), method = "complete")
plot(hc.xsc, main = "Hierarchical Clustering with Scaled Features")
```

Hierarchical Clustering with Scaled Features



lower height because standardizing reduces the raw distances between points
slightly messier dendrogram

Here, the unscaled version was actually better for this specific dataset. Unscaled data. x shift of +3, very clear separation of both groups, cleaner dendrogram Scaled: shift gets compressed, separation reduced, messier

```
h_complete_scaled <- cutree(hc.xsc,2)
h_complete_scaled
```

```
## [1] 1 1 1 2 1 1 1 2 1 1 1 1 2 1 1 1 1 1 2 1 1 1 2 2 2 2 2 2 1 2 2 2 2 2
## [39] 2 2 2 2 2 1 2 1 2 2 2 2
```

takeaway of scaled vs unscaled: unscaled performed better!
for clustering with scaled features, there are more missclassifications!

Evaluate the clustering results

In this example, we already know the true clusters. We can use ARI in package `mclust` to evaluate the clustering results. ARI: Adjusted Rand Index, a metric that measures how well the clustering matches the true labels ARI = 1 : perfect match, ARI = 0 : o better than random, ARI < 0 : worse than random

```
suppressMessages(library(mclust))
```

```
#code compares true lables (plt_x$cluster) with redicted cluster lables from
adjustedRandIndex(plt_x$cluster, h_complete) #complete linkage
```

```
## [1] 0.7697077
```

```
adjustedRandIndex(plt_x$cluster, h_average) #average
```

```
## [1] 0.7697077
```

```
adjustedRandIndex(plt_x$cluster, h_single) #single
```

```
## [1] 0
```

```
adjustedRandIndex(plt_x$cluster, h_complete_scaled) #complete with scaled data
```

```
## [1] 0.5083831
```

Which parameters showed best results?

K-means clustering

In k means clustering, we must specify the number of clusters beforehand. The function `kmeans()` performs K-means clustering in R. We begin with the same simple simulated example in previous section.

We now perform K-means clustering with $K = 2$.

```
km.out2 <- kmeans(x, 2, nstart = 20)
#x: the data we generated, 2: number of clusters k
#nstart: runs algorithm 20 times with different random starting points, keeps the best result
km.out2
```

```
## K-means clustering with 2 clusters of sizes 26, 24
##
## Cluster means:
##      [,1]      [,2]
## 1  3.329034 -0.142167
## 2 -0.337416 -0.117932
##
## Clustering vector:
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 1 2 1 2 2 2 2
##
## Within cluster sum of squares by cluster:
## [1] 62.42189 54.29051
## (between_SS / total_SS =  59.0 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

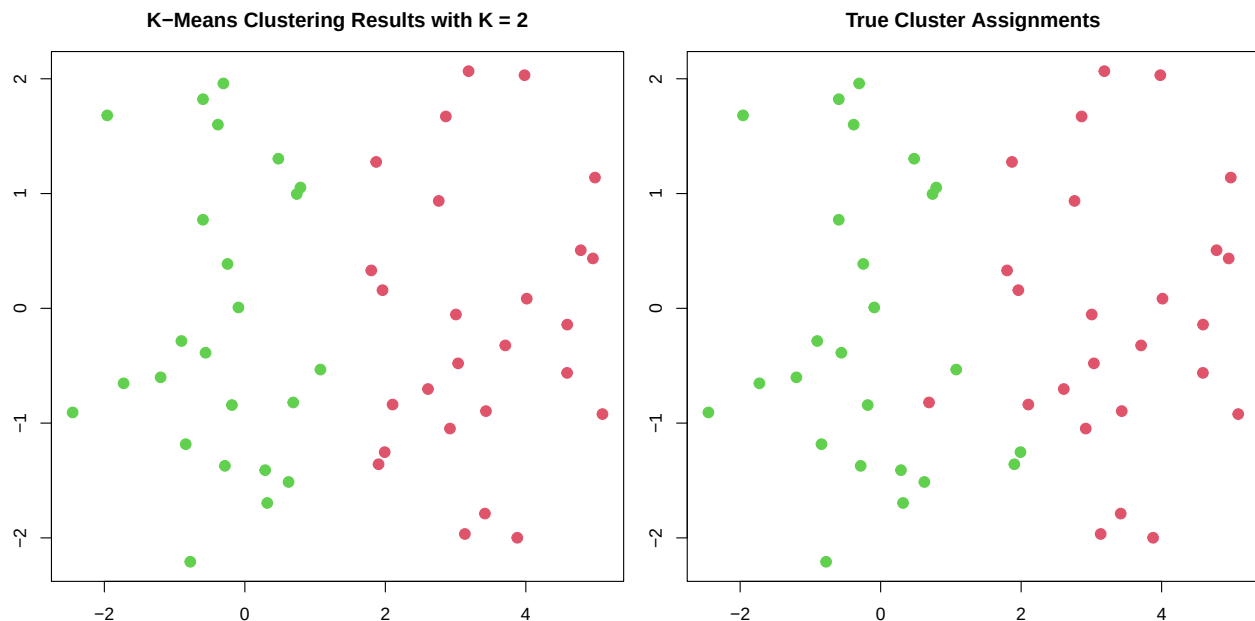
```
#Cluster sizes: 24, 26: Balanced, close to the true 25/25 split
# cluster means: centroids are exactly where we expect them: around 0;0 and 3;0
# 59% of the total variance in the data is explained by the cluster separation
# the remaining 41% is variance within the clusters themselves.
```

The cluster assignments of the 50 observations are contained in `'km.out$cluster'`.

```
km.out2$cluster #vector that contains the cluster assignments of the 50 obs.
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 1 2 1 2 2 2 2
```

```
#plot of data with each obs. colored according to its cluster assignment
par(mfrow = c(1, 2), mar = c(2, 2, 3, 1))
plot(x, col = (km.out2$cluster + 1), main = "K-Means Clustering Results with K = 2",
      xlab = "", ylab = "", pch = 20, cex = 2)
# col = (km.out2$cluster + 1) makes points green and red, pch controls point shape, cex point size
plot(x, col = (as.numeric(as.factor(plt_x$cluster)) + 1), main = "True Cluster Assignments",
      xlab = "", ylab = "", pch = 20, cex = 2)
```



```
# k means wrongly classifies 16,44,46 as well. but otherwise perfectly separated
# the observations into two clusters, although we did not train the algorithm!
# same result as for complete linkage clustering!
```

How does this compare to hierarchical clustering?

```
adjustedRandIndex(km.out2$cluster, h_complete)
```

```
## [1] 1
```


*# how similar are k means clustering and complete linkage clustering's assignments to each other?
 #in this case, both methods produce the same cluster assignments, ARI=1 means they agree perfectly*

```
set.seed(1)
#we now run k-means with k=3 on our data, which only has two true clusters.
# lets see how it performs:
km.out3 <- kmeans(x, 3)
km.out3
```

```
## K-means clustering with 3 clusters of sizes 10, 21, 19
##
## Cluster means:
##      [,1]      [,2]
## 1  2.0426526  1.1821377
## 2 -0.4812553 -0.2943996
## 3  3.5861438 -0.6402999
##
## Clustering vector:
## [1] 3 1 3 1 3 3 3 1 3 1 3 1 3 1 3 2 3 3 3 3 3 1 3 3 3 2 1 2 1 2 1 2 2 2 2 2 2
## [39] 2 2 2 2 2 3 2 3 2 2 2 2
##
## Within cluster sum of squares by cluster:
## [1] 15.71096 45.47255 32.74425
## (between_SS / total_SS =  67.0 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

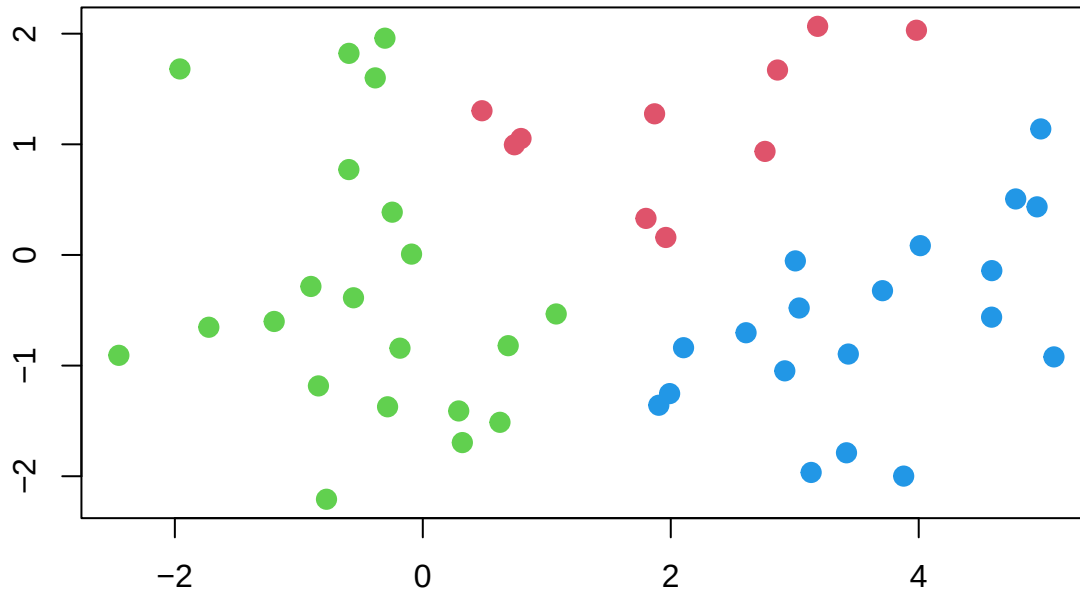
K-means with k=3 on this data splits the data into three uneven groups: 10,21,19 Group 2 stayed intact (x and y around 0) Group 1 (around x = 3) got split into an upper (new cluster 1) and lower (new cluster 3) group

WSS: Cluster 2 has the highest WSS — it's the largest and most spread out group

67% of variance is explained by the clusters (59% for k=2) k=3 “explains” more, but only because it artificially splits a real group, creating fake separation More clusters don't always mean better clustering if the extra clusters are artificial. -> Choosing the right k is crucial!

```
plot(x, col = (km.out3$cluster + 1),
main = "K-Means Clustering Results with K = 3", xlab = "", ylab = "", pch = 20, cex = 2)
```

K-Means Clustering Results with K = 3



which cluster was assigned which color?

Choosing the number of clusters k

Silhouette index is a widely used method to evaluate clustering results. For each point $i \in C_I$, where C_I is the cluster I , we first define

$$a(i) = \frac{1}{|C_I - 1|} \sum_{j \in C_I, i \neq j} d(i, j)$$

-> the mean distance between i and all other data points in the same cluster;

and

$$b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} d(i, j)$$

-> mean distance to members of the next closest cluster

The silhouette (value) of one data point i is defined as

$$s(i) = \frac{b(i) - a(i)}{\max\{b(i), a(i)\}}$$

numerator measures: how much better does this point fit in its own cluster compared to the next best cluster
denominator just picks whichever is larger between $a(i)$ and $b(i)$ and divides by it -> normalization!

We can also get the average silhouette width when using other algorithms:

With a hierarchical method (Complete linkage here):

```
suppressWarnings(suppressMessages(library(cluster)))
dist.x <- dist(x) #computes and saves distance matrix
x.complete.link <- hclust(dist.x, method='complete') # complete linkage dendrogram

summary(silhouette(cutree(x.complete.link, k=2), dist(dist.x)))$avg.width
```

```
## [1] 0.4914547
```

*#cutree cuts dendrogram into 2 clusters and computes $s(i)$ for every point,
then computes average silhouette width as one number*

#same procedure for $k = 3, 4, 5$:

```
summary(silhouette(cutree(x.complete.link, k=3), dist(dist.x)))$avg.width
```

```
## [1] 0.354211
```

```
summary(silhouette(cutree(x.complete.link, k=4), dist(dist.x)))$avg.width
```

```
## [1] 0.2501386
```

```
summary(silhouette(cutree(x.complete.link, k=5), dist(dist.x)))$avg.width
```

```
## [1] 0.2959878
```

Rule: pick k with the highest average silhouette width. Here: $k = 2$, silhouette width = 0.49

With k-means:

```
summary(silhouette(kmeans(x, centers=2, iter.max=100, nstart=25)$cluster, dist(x)))$avg.width
```

```
## [1] 0.482954
```

```
summary(silhouette(kmeans(x, centers=3, iter.max=100, nstart=25)$cluster, dist(x)))$avg.width
```

```
## [1] 0.4252862
```

```
summary(silhouette(kmeans(x, centers=4, iter.max=100, nstart=25)$cluster, dist(x)))$avg.width
```

```
## [1] 0.3849287
```

```
summary(silhouette(kmeans(x, centers=5, iter.max=100, nstart=25)$cluster, dist(x)))$avg.width
```

```
## [1] 0.4158412
```

```
# same thing, k=2 with 0.483 highest average silhouette width
```

In each case, we choose the value of k associated with the LARGEST average silhouette width.

Spectral clustering

Let's look at how to implement spectral clustering and compare the results to a k-means approach.

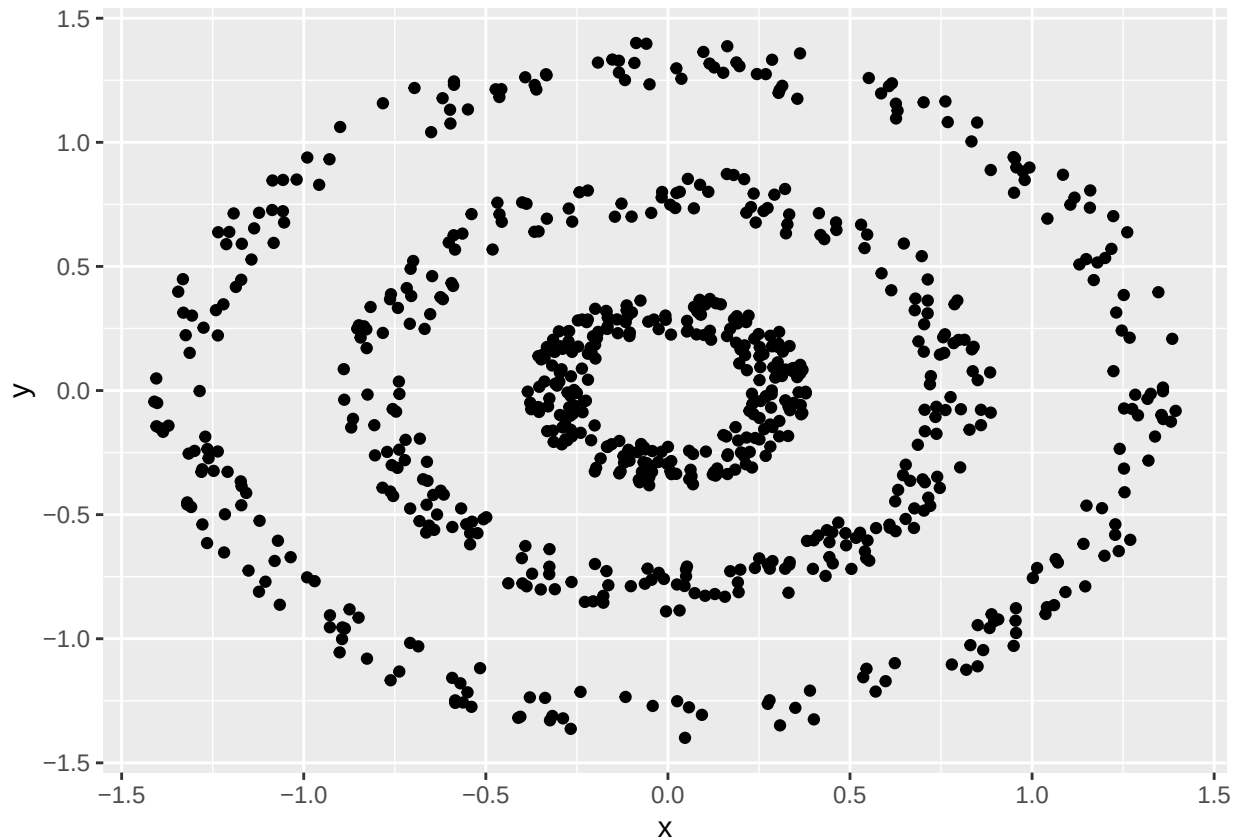
```
## Packages  
library(tidyverse)  
library(gridExtra)
```

```
##  
## Attaching package: 'gridExtra'  
  
## The following object is masked from 'package:dplyr':  
##  
##      combine
```

```
library(ggplot2)
```

We first need to generate some data for clustering. We will generate three circular, uniform distributions and plot them:

```
# Generate circular uniform distributions of data  
set.seed(1748)  
  
## Set min and max parameters for each distribution  
min_vals <- c(0.05, 0.5, 1.5)  
max_vals <- c(0.15, 0.8, 2.0)  
x <- c()  
y <- c()  
  
## Generate distributions using parameters  
for(i in 1:3){  
  r <- sqrt(runif(250, min = min_vals[i], max = max_vals[i]))  
  p <- runif(250)  
  theta <- p * 2 * pi  
  x_temp <- r * cos(theta)  
  y_temp <- r * sin(theta)  
  x <- c(x, x_temp)  
  y <- c(y, y_temp)  
}  
  
## Create data frame  
data <- as.data.frame(cbind(x, y))  
  
## Plot coordinates  
ggplot(data=data, aes(x=x, y=y)) + geom_point()
```



```
#result: 750 points total (250 per ring), arranged in 3 concentric rings
# clearly 3 clusters by eye
```

Our goal is to cluster the data. By sight, there are clearly three rings representing the three distributions of data. We will cluster the data using k-means first:

```
# K-means Application
set.seed(1748)

## Run Kmeans algorithm with 3 clusters (sounds plausible as we have 3 rings)
km <- kmeans(data, 3)

## Retrieve and store cluster assignments
kmeans_clusters <- km$cluster
kmeans_results <- cbind(data, cluster = as.factor(kmeans_clusters))
head(kmeans_results)
```

```
##           x           y cluster
## 1  0.2683618260 -0.02650468      3
## 2  0.3785791290 -0.01142605      3
## 3  0.0001506089 -0.22682784      3
## 4 -0.2894357939 -0.14823834      1
## 5  0.0990621659  0.22378948      2
## 6 -0.1655848202  0.25820211      2
```

The output above represents the cluster each point is assigned to using k-means. We will now calculate the cluster assignments using spectral clustering:

```

# Spectral Clustering

## Create Similarity matrix of Euclidean distance between points
S <- as.matrix(dist(data))

## Create Degree matrix
D <- matrix(0, nrow=nrow(data), ncol = nrow(data)) # empty nxn matrix

for (i in 1:nrow(data)) {

  # Find top 10 nearest neighbors using Euclidean distance
  index <- order(S[i,])[2:11]

  # Assign value to neighbors
  D[i,][index] <- 1
}

# find mutual neighbors
D = D + t(D)
D[ D == 2 ] = 1

# find degrees of vertices
degrees = colSums(D)
n = nrow(D)

## Compute Laplacian matrix
# Since k > 2 clusters (3), we normalize the Laplacian matrix:
laplacian = ( diag(n) - diag(degrees^(-1/2)) %*% D %*% diag(degrees^(-1/2)) )

## Compute eigenvectors
eigenvectors = eigen(laplacian, symmetric = TRUE)
n = nrow(laplacian)
eigenvectors = eigenvectors$vectors[, (n - 2):(n - 1)]

set.seed(1748)
## Run Kmeans on eigenvectors
sc = kmeans(eigenvectors, 3)

## Pull clustering results
sc_results = cbind(data, cluster = as.factor(sc$cluster))
head(sc_results)

```

```

##           x           y cluster
## 1  0.2683618260 -0.02650468      1
## 2  0.3785791290 -0.01142605      1
## 3  0.0001506089 -0.22682784      1
## 4 -0.2894357939 -0.14823834      1
## 5  0.0990621659  0.22378948      1
## 6 -0.1655848202  0.25820211      1

```

The output above represents the cluster each point is assigned to using spectral clustering. Let's now graph the data and use colors to represent the cluster assignment of both k-means and spectral clustering:

```
# Kmeans plot
kmeans_plot = ggplot(data = kmeans_results, aes(x=x, y=y, color = cluster)) +
  geom_point() +
  scale_color_manual(values = c('1' = "violetred2",
                                '2' = "darkorchid2",
                                '3' = "darkolivegreen2")) +
  ggtitle("K-Means Clustering") +
  theme(panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
        panel.background = element_blank(), axis.line = element_line(colour = "black"),
        plot.title = element_text(hjust = 0.5), legend.position="bottom")

# Spectral Clustering plot
sc_plot = ggplot(data = sc_results, aes(x=x, y=y, color = cluster)) +
  geom_point() +
  scale_color_manual(values = c('1' = "violetred2",
                                '2' = "darkorchid2",
                                '3' = "darkolivegreen2")) +
  ggtitle("Spectral Clustering") +
  theme(panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
        panel.background = element_blank(), axis.line = element_line(colour = "black"),
        plot.title = element_text(hjust = 0.5), legend.position="bottom")

# Arrange plots
grid.arrange(kmeans_plot, sc_plot, nrow=1)
```

